A Time-of-Day-Aware Empirical Study of AWS Spot Instance Reliability for Short-Running Workloads

Nagisa Nasu*[1,2], Atsuya Kanno[1,3], Akihito Kohiga[1,4], and Takahiro Koita[1,5]

[1] Graduate School of Science and Engineering,
Doshisha University, Kyotanabe, Kyoto, 610-0394, Japan
[2] ctwn0144@mail4.doshisha.ac.jp
[3] ctwp0167@mail4.doshisha.ac.jp
[4] akohiga@mail.doshisha.ac.jp
[5] tkoita@mail.doshisha.ac.jp

Abstract. Spot instances reduce cloud computing costs but may be interrupted before short-running workloads complete. Interruptions often result in a full retry and increased execution time and cost. Existing indicators such as Spot Placement Score (SPS) assess launch-time availability but not post-launch runtime continuity. We define a deadline-conditioned completion feasibility metric $F_{\mathrm{comp}}(d)$—the fraction of launched instances remaining uninterrupted until deadline d—and measure it across four AWS regions and three instance types over one week, for deadlines of 5, 10, 30, and 60 minutes. $F_{\mathrm{comp}}(d)$ varied substantially by local time period for compute-oriented and GPU-based types, with the Night period achieving the highest feasibility. These findings suggest time of day is an operationally meaningful scheduling factor for short-running spot workloads.

Keywords: Cloud Computing, Interruption Event, CI/CD, Completion Feasibility, Workload Scheduling, Diurnal Pattern
Submission type: Short Research Paper

1 Introduction

1.1 Background

In recent years, cloud computing has been widely adopted by enterprises, research institutions, and individual developers. As the scale of cloud usage increases and the demand for high-performance computing resources grows, cloud computing costs have become an important issue for many users.

One approach to reducing cloud computing costs is the use of spot instances. Spot instances are a pricing model in which cloud providers offer spare computing resources at lower prices than on-demand instances [1]. Compared with on-demand instances, spot instances can be used at a discount of several tens of

* Contact Author.

percent to up to approximately 90% [2]. Therefore, spot instances are used not only for large-scale computational workloads such as machine learning and batch processing, but also for workloads that temporarily require computing resources, such as automated build and test processes in CI/CD [3].

However, because spot instances rely on spare capacity, they may be interrupted when cloud demand increases or available capacity decreases. Such an interruption is called an interruption event. If an interruption event occurs during the execution of a workload, the running process may stop before completion. In particular, when batch processing, machine learning, or CI/CD build and test processes are interrupted, the workload may need to be re-executed. As a result, both execution time and cloud usage cost may increase.

Thus, although spot instances are effective for reducing cloud computing costs, their reliability is affected by interruption events. Existing approaches include bidding strategies and provider-specific indicators such as Spot Placement Score (SPS) [4, 5], which estimates whether a spot request is likely to be fulfilled. However, for users of short-running workloads such as CI/CD pipelines and batch jobs, the key question is not only whether an instance can be launched, but also whether it will remain available until the workload completes. Although prior studies have shown that spot availability may vary with time of day [6], whether this variation can guide completion-oriented scheduling remains insufficiently addressed. In this study, we therefore treat time of day as an operational scheduling factor and evaluate spot instance reliability in terms of deadline-conditioned completion feasibility.

1.2 Contributions

The novelty of this study is not merely in analyzing interruption tendencies of spot instances. While prior studies have treated SPS and survival time mainly as infrastructure-level availability metrics, this study treats time of day as an operational variable for deciding when to execute short-running workloads. We evaluate deadline-conditioned completion feasibility for workload deadlines of 5, 10, 30, and 60 minutes based on directly observed launch and interruption behavior.

The main contributions of this paper are as follows:

- We reformulate spot instance reliability for short-running workloads as deadline-conditioned completion feasibility, rather than only as launch availability or infrastructure-level survival time.
- We analyze time-of-day effects on short-workload completion feasibility, treating time of day as an operational scheduling factor for CI/CD and short batch-processing workloads.
- We implement an AWS-based measurement framework that separately records launch success and runtime interruption behavior across multiple regions and instance types.
- We conduct a preliminary empirical study using workload deadlines of 5, 10, 30, and 60 minutes to compare completion feasibility by time of day, region, and instance type.

2 Related Work

2.1 Spot Instance Availability and Temporal Variation

Prior work has addressed spot instance reliability through approaches such as checkpointing, migration, and scheduling [2]. SPS is a provider-specific AWS indicator that estimates the likelihood a spot request can be fulfilled under given conditions [7]. Katayama et al. [5] showed that SPS can guide migration destination selection, improving launch-time availability. Kim et al. [6] showed that SPS varies periodically with time of day and day of week and correlates with survival time, treating temporal variation primarily as a characteristic of spot markets. However, neither SPS nor survival time directly indicates whether a launched instance will remain available long enough for a given workload to complete, and they do not provide a workload-deadline-oriented view of time-of-day effects.

2.2 Spot Instance Datasets and Workload-Oriented Use

Lee et al. [8] proposed SpotLake, showing that GPU and large instance types tend to have lower spot availability and that availability differs across regions. Garg et al. [4] and Ramesh and Lennon [3] considered spot instances for cost reduction and DevOps workflows. However, the relationship between time of day and completion feasibility of short-running workloads remains insufficiently studied. In contrast, this paper focuses on deadline-conditioned completion feasibility, separating launch-time availability from runtime continuity to analyze spot instance usage from the viewpoint of short-workload execution timing.

3 Proposed Measurement Framework

3.1 Problem Definition

Existing indicators such as SPS characterize whether a spot request can be fulfilled. They do not indicate whether a launched instance will remain available long enough for a workload to complete. For short-running workloads such as CI/CD pipelines and batch jobs, these are distinct concerns: even a successfully launched instance may be interrupted before the workload finishes, forcing a full retry and increasing both execution time and cost. Users of such workloads therefore need a deadline-oriented view of spot instance reliability, not just request fulfillment likelihood.

Furthermore, many short-running workloads can be scheduled flexibly—for example, during nighttime or low-demand periods. Time of day is therefore a potentially actionable factor for reducing interruption risk. However, existing indicators do not expose the time-of-day effect in deadline-oriented terms. SPS and $F_{\mathrm{comp}}(d)$ thus capture complementary aspects of spot instance reliability: SPS addresses launch-time placement likelihood, whereas $F_{\mathrm{comp}}(d)$ addresses post-launch runtime continuity for workload-specific deadlines.

3.2 Objective

This study measures deadline-conditioned completion feasibility—the probability that a successfully launched spot instance remains uninterrupted until a workload deadline of 5, 10, 30, or 60 minutes—and analyzes how it varies by time of day, region, and instance type. Based on the results, we discuss practical scheduling strategies for short-running workloads on AWS Spot Instances.

3.3 Approach

This study does not execute actual application-level workloads. Instead, we use uninterrupted instance availability until a given deadline as a proxy for workload completion feasibility: if a successfully launched instance is not interrupted before deadline d, we regard it as feasible for a workload with that execution-time requirement to complete under the observed condition.

Figure 1 shows the proposed measurement framework. The framework periodically submits spot instance launch requests using a scheduler and collects launch results, runtime status, and interruption status.

First, the scheduler submits spot instance launch requests for each target instance type at regular intervals. Each successfully launched instance is observed for a fixed period of one hour. During this observation window, the system records whether the spot request is successfully fulfilled, how long the instance remains available, and whether an interruption event occurs before the scheduled termination. If no interruption event occurs within the observation window, the instance is terminated by the system.

In this study, we use three metrics: launch success rate, interruption-before-deadline rate, and observed availability duration. The launch success rate is defined as the ratio of successfully launched instances to the total number of spot instance launch requests. The interruption-before-deadline rate is calculated for workload deadlines of 5, 10, 30, and 60 minutes, and represents the ratio of successfully launched instances that are interrupted before each deadline. The observed availability duration represents the duration for which successfully launched instances remain available within the one-hour observation window.

Let N_{req} be the number of spot instance launch requests, N_{succ} be the number of successfully launched instances, and $N_{\mathrm{int}}(d)$ be the number of successfully launched instances that are interrupted before workload deadline d. The launch success rate R_{launch} and the interruption-before-deadline rate $R_{\mathrm{int}}(d)$ are defined as follows:

$$R_{\mathrm{launch}} = \frac{N_{\mathrm{succ}}}{N_{\mathrm{req}}} \tag{1}$$

$$R_{\mathrm{int}}(d) = \frac{N_{\mathrm{int}}(d)}{N_{\mathrm{succ}}} \tag{2}$$

Here, d represents the workload deadline. In this study, we use $d = 5, 10, 30,$ and 60 minutes to evaluate whether spot instances can support short-running

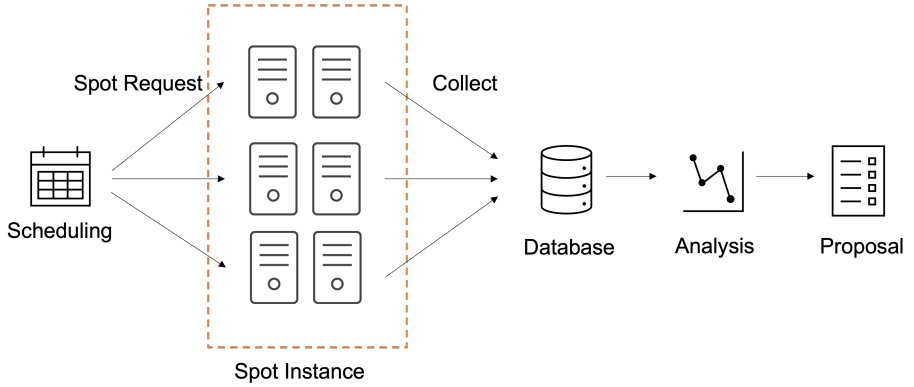


Fig. 1. Proposed measurement framework

workloads with different execution-time requirements. We also define the deadline-conditioned completion feasibility $F_{\mathrm{comp}}(d)$ as follows:

$$F_{\mathrm{comp}}(d) = 1 - R_{\mathrm{int}}(d) \tag{3}$$

This metric represents the fraction of successfully launched instances that remained available until deadline d. It should be interpreted as a survival-based proxy for workload completion feasibility, not as a measured success rate of actual application-level workloads.

By analyzing these metrics by local time of day, region, and instance type, this study aims to clarify the conditions under which spot instances are more likely to be launched successfully and remain available long enough for short-running workloads to complete. Based on the obtained results, we discuss a spot instance usage strategy that considers time periods, regions, and instance types with relatively higher short-workload completion feasibility.

4 Experiments

4.1 Experimental Methods

In this experiment, we implement the proposed measurement framework on AWS using a scheduler-driven Lambda function that periodically launches spot instances and records launch results, runtime status, and interruption events in a database.

We selected three instance types—c6a.large (compute-oriented), g4dn.xlarge (GPU-based), and t3.micro (lightweight)—as representative examples of different workload categories [9, 10], to compare interruption tendencies among instance types with different resource characteristics.

The experiment targets four AWS regions: N. Virginia (us-east-1), Tokyo (ap-northeast-1), Ireland (eu-west-1), and Singapore (ap-southeast-1). In each

region, one spot instance is launched every hour for each instance type. Each instance is observed for up to one hour and is terminated by the system if no interruption event occurs during the observation window.

Timestamps are normalized to the local time of each target region when analyzing time-of-day trends. This normalization allows us to compare interruption tendencies with respect to the local demand cycle of each region. We divide each local day into four six-hour periods: Night (00:00–05:59), Morning (06:00–11:59), Afternoon (12:00–17:59), and Evening (18:00–23:59). Local time is determined according to the time zone of each AWS region. No maximum spot price was specified in the spot instance requests. Therefore, the experiment observes spot instance behavior under the default AWS spot pricing configuration.

Based on the collected information, we calculate the launch success rate, the interruption-before-deadline rate for 5, 10, 30, and 60 minutes, the deadline-conditioned completion feasibility for each deadline, and the observed availability duration for each combination of time period, region, and instance type. The interruption-before-deadline rate and deadline-conditioned completion feasibility are calculated only for successfully launched instances, while the launch success rate is calculated based on all spot instance launch requests. This separation allows us to distinguish between failures caused by insufficient spot capacity at launch time and interruptions that occur after an instance has started running.

The experiment was conducted from May 19, 2026, to May 25, 2026. Because the observation period was limited to one week, the obtained results may be affected by short-term fluctuations in spot capacity and may not fully represent long-term, weekly, or seasonal trends. In addition, since only one spot instance was launched every hour for each combination of region and instance type, the number of samples for each time period is limited. Therefore, the results of this experiment should be interpreted as preliminary empirical observations under the given experimental conditions, rather than as universally generalizable interruption trends.

To distinguish spot interruptions from normal system termination, an instance that becomes unavailable before its scheduled one-hour observation end without being terminated by the measurement system is treated as an interruption candidate. Collected records are grouped by local time of day, region, and instance type to calculate the metrics defined in Section 3.

5 Results

5.1 Overview

Over the observation period, a total of $N_{\mathrm{req}} = 2{,}016$ spot instance launch requests were submitted across four regions and three instance types. Of these, $N_{\mathrm{succ}} = 1{,}950$ instances were successfully launched, yielding an overall launch success rate of $R_{\mathrm{launch}} = 96.7\%$. Among the successfully launched instances, 117 (6.0%) were interrupted before the end of the one-hour observation window.

Figure 2 shows $F_{\mathrm{comp}}(60)$ aggregated by local time period, region, and instance type. The t3.micro instance type achieved $F_{\mathrm{comp}}(60) \geq 0.95$ across all



Fig. 2. $F_{\mathrm{comp}}(60)$ heatmap by local time period, region, and instance type. Annotations show the value and sample counts ($n_{\mathrm{comp}}/n_{\mathrm{succ}}$).

regions and time periods, indicating that lightweight instances are relatively robust to interruption regardless of execution timing. In contrast, c6a.large and g4dn.xlarge exhibited larger time-period variation, with values ranging from approximately 0.65 to 1.00. This suggests that time of day is more operationally relevant for compute-oriented and GPU-based instance types.

5.2 Time-of-Day Effects on Deadline-Conditioned Completion Feasibility

Figure 3 shows $F_{\mathrm{comp}}(d)$ for $d \in \{5, 10, 30, 60\}$ minutes for the four region–instance-type combinations with the largest observed time-period variation in $F_{\mathrm{comp}}(60)$.

In panel (a), N. Virginia c6a.large, the Night period achieved $F_{\mathrm{comp}}(60) = 0.98$ (95% CI: 0.88–1.00; $n = 42$), whereas the Morning period fell to $F_{\mathrm{comp}}(60) = 0.69$ (95% CI: 0.54–0.81; $n = 39$), a difference of approximately 0.29. The two confidence intervals do not overlap, indicating that this gap is unlikely to be attributable to sampling variation. Even at the shortest deadline of $d = 5$ minutes, the Morning period already showed $F_{\mathrm{comp}}(5) = 0.90$ (95% CI: 0.76–0.96), indicating that interruption risk begins to accumulate within the first five minutes. The Night and Evening periods maintained $F_{\mathrm{comp}}(d) \geq 0.90$ across all deadlines.

Panel (b), Ireland c6a.large, shows a qualitatively similar pattern. The Night period achieved $F_{\mathrm{comp}}(60) = 1.00$ (95% CI: 0.92–1.00; $n = 42$) for all deadlines, while the Morning period dropped to $F_{\mathrm{comp}}(60) = 0.73$ (95% CI: 0.58–0.84; $n = 41$) and the Afternoon to 0.88 (95% CI: 0.75–0.95; $n = 42$). These results suggest that c6a.large instances in both N. Virginia and Ireland are more susceptible to interruption during daytime hours.

For g4dn.xlarge in Tokyo (panel (c)), the Morning period showed the largest degradation with $F_{\mathrm{comp}}(60) = 0.77$ (95% CI: 0.59–0.88; $n = 30$), while the Night period reached $F_{\mathrm{comp}}(60) = 1.00$ (95% CI: 0.92–1.00; $n = 42$) and the Evening period reached $F_{\mathrm{comp}}(60) = 0.97$ (95% CI: 0.85–0.99; $n = 35$). In Singapore (panel (d)), the Afternoon period exhibited the lowest $F_{\mathrm{comp}}(60) = 0.78$ (95%

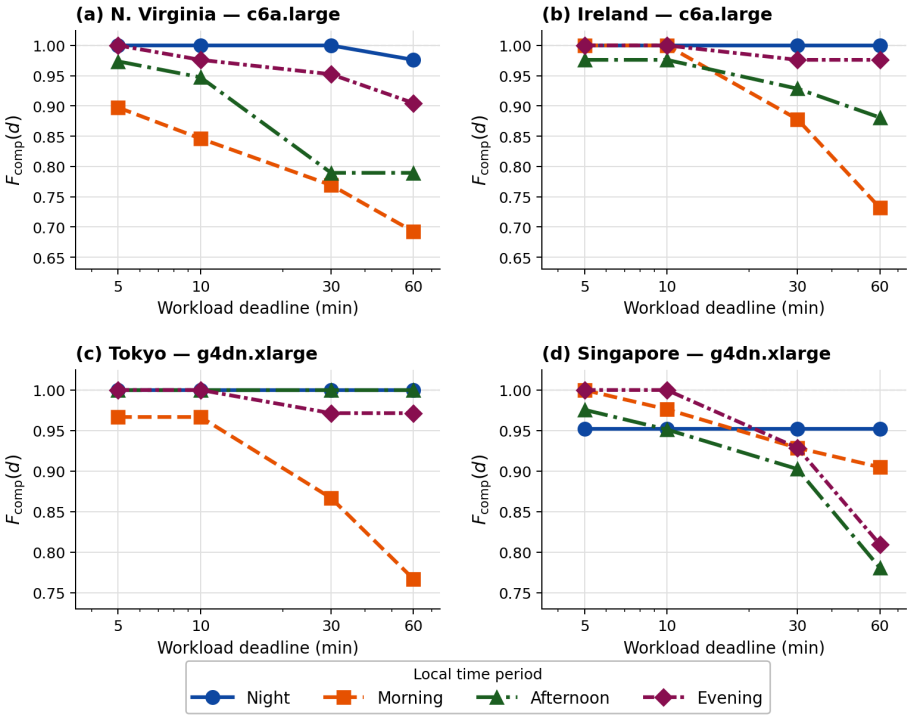


Fig. 3. $F_{\mathrm{comp}}(d)$ for the four region–instance-type combinations with the largest time-period variation. Lines represent local time periods; the x-axis uses a logarithmic scale.

CI: 0.63–0.88; $n = 41$), whereas the Morning was also reduced at 0.90 (95% CI: 0.78–0.96; $n = 42$) compared to Night at 0.95 (95% CI: 0.84–0.99; $n = 42$).

Across all four panels, the Night period consistently achieved the highest or near-highest $F_{\mathrm{comp}}(d)$ values, while the Morning or Afternoon period was associated with the greatest interruption risk depending on the region. These observations provide preliminary evidence that time of day can serve as an operationally meaningful indicator for scheduling short-running workloads on spot instances.

6 Discussion

6.1 Time-of-Day as an Operational Scheduling Factor

The results show that $F_{\mathrm{comp}}(d)$ varies substantially by local time period for compute-oriented and GPU-based instance types, while remaining largely stable for t3.micro. The largest gap (approximately 0.29, with non-overlapping confidence intervals) was observed in N. Virginia c6a.large between the Night and Morning periods, with qualitatively similar contrasts in Ireland c6a.large and

Tokyo g4dn.xlarge. These findings suggest that, for workloads with flexible execution timing, selecting the Night period (00:00–05:59 local time) can substantially reduce interruption risk before the workload deadline.

The observed pattern is consistent with the hypothesis that local cloud demand follows a diurnal cycle. During local nighttime hours, business-driven demand for compute and GPU capacity is likely lower, leaving more spare capacity available for spot instances. Conversely, the Morning or Afternoon periods—corresponding to the start of local business hours—appear to be associated with increased spot market competition and higher interruption rates. The regional difference in which period is riskiest further supports this interpretation: N. Virginia and Ireland show Morning as the riskiest period, which aligns with the start of the North American and European business day; Singapore shows Afternoon as riskiest, consistent with peak activity during Asian afternoon hours.

6.2 Instance Type and Interruption Susceptibility

The observed robustness of t3.micro ($F_{\text{comp}}(60) \geq 0.95$ in all conditions) contrasts with the higher interruption susceptibility of c6a.large and g4dn.xlarge. This difference may be attributed to the relative demand for each instance type in the spot market. Burstable lightweight instances such as t3.micro are likely to face less competition for spare capacity, whereas compute-optimized and GPU-accelerated instances are in higher demand for machine learning and batch workloads. This is consistent with the findings of Lee et al. [8], who reported that GPU instances and large instance types tend to have lower spot availability. Our results extend this finding to the deadline-conditioned perspective: not only are GPU and compute instances more likely to be interrupted, but their interruption risk varies more strongly with time of day than that of lightweight instances.

6.3 Implications for Workload Scheduling

These preliminary observations have direct implications for practitioners who use spot instances for short-running workloads with flexible scheduling. For c6a.large or g4dn.xlarge instances in regions such as N. Virginia, Ireland, or Tokyo, scheduling CI/CD jobs or batch-processing tasks during the Night period can increase $F_{\text{comp}}(60)$ by up to 0.29 compared to the Morning period, without requiring any changes to the workload itself or any provider-specific API calls. Since $F_{\text{comp}}(d)$ is derived solely from directly observed launch and interruption records, this approach provides a transparent and interpretable scheduling signal derived from directly observed interruption behavior. While this study does not empirically compare the proposed metric with SPS, $F_{\text{comp}}(d)$ captures a different aspect of spot reliability: post-launch runtime continuity for workload-specific deadlines. These results should not be interpreted as a replacement for SPS: the two metrics address different stages of spot instance reliability, and users may benefit from consulting both when selecting launch conditions and execution timing.

It should be noted that the riskiest time period differs by region: Morning in N. Virginia, Ireland, and Tokyo, whereas Afternoon in Singapore. Users should therefore derive $F_{\mathrm{comp}}(d)$ from their own observed data in the target region rather than applying a single universal rule.

When a workload must run at a fixed wall-clock time, region selection can substitute for time-shifting: routing the job to a region where it is currently nighttime exploits the lower interruption rates of the Night period without delaying execution.

6.4 Limitations

Several limitations must be acknowledged. First, the observation period is limited to one week, and the sample size per time period is approximately 30–42 instances per region–instance-type combination. The results may therefore reflect short-term fluctuations in spot capacity and should not be generalized as long-term or seasonal trends. Second, this study uses uninterrupted instance availability as a proxy for workload completion feasibility and does not execute actual application-level workloads. Application-specific factors such as checkpointing, startup overhead, and inter-job dependencies are outside the scope of this study. Third, only one instance was launched per hour for each combination, so concurrent-launch effects and queue behavior under high demand are not captured. Fourth, no maximum spot price was specified, so the results reflect behavior under the default AWS pricing configuration only. Future work should address these limitations by extending the observation period, increasing the sampling frequency, running actual CI/CD workloads, and comparing $F_{\mathrm{comp}}(d)$ against SPS and other provider-specific indicators.

7 Conclusion

This paper defined deadline-conditioned completion feasibility $F_{\mathrm{comp}}(d)$ as a time-of-day-aware metric and empirically measured it across four AWS regions and three instance types to evaluate the completion feasibility of short-running workloads on spot instances. Unlike indicators that address only launch-time availability or infrastructure-level survival time, $F_{\mathrm{comp}}(d)$ captures post-launch runtime continuity for workload-specific deadlines of 5, 10, 30, and 60 minutes, with launch success and interruption behavior recorded separately.

The main idea of this study is to treat time of day as an operational variable for deciding when to execute short-running workloads on spot instances. By analyzing launch success rate, interruption-before-deadline rate, deadline-conditioned completion feasibility, and observed availability duration by local time of day, region, and instance type, the proposed approach supports a more interpretable spot instance usage strategy based on directly observed completion behavior. Future work will extend the measurement period, increase the sample size, evaluate actual CI/CD and batch-processing workloads, and compare the observed completion feasibility with provider-specific indicators such as SPS.

References

1. Amazon Web Services: Amazon EC2 Spot Instances. https://aws.amazon.com/jp/ec2/spot/ (Accessed: May 2026)
2. Lin, L., Pan, L., Liu, S.: Methods for improving the availability of spot instances: A survey. Computers in Industry, 141, 103718 (2022). https://doi.org/10.1016/j.compind.2022.103718
3. Ramesh, D.G.A., Lennon, R.G.: Evaluating the Impact of Reserved Instances vs. Spot Instances in DevOps Cloud Workflows. In: 2025 35th Irish Signals and Systems Conference (ISSC). IEEE (2025). https://doi.org/10.1109/ISSC67739.2025.11291505
4. Garg, D., Kaata, S.K., Bhumireddy, D.R., Bolla, B.K.: Reducing Cloud Cost Through Optimized Bidding for AWS Spot Instances. In: 2025 3rd International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT). IEEE (2025). https://doi.org/10.1109/ICAICCIT68829.2025.11433986
5. Katayama, D., Kasai, K., Koita, T.: Migration Destination Selection Algorithm for Spot Instances using SPS. In: 2022 IEEE International Conference on Big Data (Big Data), pp. 6690–6692. IEEE, Osaka (2022). https://doi.org/10.1109/BigData55660.2022.10020520
6. Kim, K., Lee, K.: Making Cloud Spot Instance Interruption Events Visible. In: Proceedings of the ACM Web Conference 2024 (WWW '24), pp. 2998–3009. ACM, New York (2024). https://doi.org/10.1145/3589334.3645548
7. Amazon Web Services: Spot Placement Score. Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-placement-score.html (Accessed: May 2026)
8. Lee, S., Hwang, J., Lee, K.: SpotLake: Diverse Spot Instance Dataset Archive Service. In: 2022 IEEE International Symposium on Workload Characterization (IISWC), pp. 242–255. IEEE (2022). https://doi.org/10.1109/IISWC55918.2022.00029
9. Pham, T.-P., Ristov, S., Fahringer, T.: Performance and Behavior Characterization of Amazon EC2 Spot Instances. In: 2018 IEEE 11th International Conference on Cloud Computing (CLOUD). IEEE (2018). https://doi.org/10.1109/CLOUD.2018.00017
10. Amazon Web Services: Amazon EC2 Instance Types. Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-types.html (Accessed: May 2026)